UNIT V

Introduction to Data Science

Functional programming · JSON and XML in Python · NumPy · Pandas · Matplotlib basics · Sample Programs 24–30

➤ What you will learn in this unit

- Functional-style Python: **lambda, map, filter, reduce** and comprehensions.
- Exchanging data using **JSON** and **XML** with the json and ElementTree modules.
- Fast numerical work with **NumPy arrays**, and table-style analysis with **Pandas** DataFrames.
- Plotting data with **Matplotlib** and solutions of sample programs **24 to 30** with real output.

1. What is Data Science?

Data Science is the process of collecting data, cleaning it, analysing it and turning it into useful decisions. Python is the most popular language for it because of its simple syntax and powerful libraries.



Python tools: json / csv → Pandas → NumPy → Matplotlib

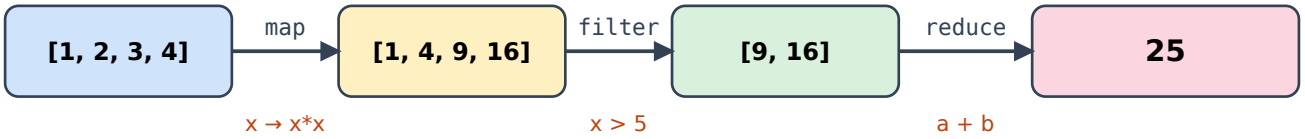
Library	Purpose
NumPy	Fast multi-dimensional arrays and mathematics
Pandas	Tables (DataFrames), data cleaning, selection and analysis
Matplotlib	Graphs and charts
json / xml	Reading and writing JSON and XML data formats

2. Functional Programming in Python

Functional programming is a style where a program is built by combining **functions**. Functions are treated as values: they can be stored in variables, passed to other functions and returned from functions.

- **Pure function** — same input always gives same output and changes nothing outside (no side effects).
- **First-class functions** — functions can be assigned to variables and passed as arguments.
- **Higher-order function** — a function that takes another function as argument (map, filter, sorted with key).
- **Lambda** — a small **anonymous** one-line function: `lambda args: expression`

Tool	What it does	Example
`map(f, seq)`	Applies f to **every** item	`map(lambda x: x*x, [1,2,3])` → 1,4,9
`filter(f, seq)`	Keeps items for which f is **True**	`filter(lambda x: x>2, [1,2,3])` → 3
`reduce(f, seq)`	Combines all items into **one value** (functools)	`reduce(lambda a,b: a+b, [1,2,3])` → 6
`[x for x in s if c]`	List comprehension — short form of map + filter	`[x*x for x in s if x%2]`
`zip(a, b)`	Pairs items of two sequences	`zip([1,2],["a","b"])`




```
from functools import reduce
nums = [1, 2, 3, 4, 5]

square = lambda x: x * x                        # lambda function
print(list(map(square, nums)))                  # map
print(list(filter(lambda x: x % 2 == 0, nums)))# filter
print(reduce(lambda a, b: a + b, nums))         # reduce
print([x * x for x in nums if x % 2 == 1])      # comprehension
print(sorted(["pear", "fig", "apple"], key=len))   # function as argument

def squares(n):                                 # generator: yields values lazily
    for i in range(n):
        yield i * i
print(list(squares(5)))
```

Output

```
[1, 4, 9, 16, 25]
[2, 4]
15
[1, 9, 25]
['fig', 'pear', 'apple']
[0, 1, 4, 9, 16]
```

i Remember

- In Python 3, *map()* and *filter()* return iterators — wrap them in *list()* to see the values.

3. JSON in Python

JSON (JavaScript Object Notation) is a lightweight text format for exchanging data (web APIs, config files). It looks like a Python dictionary. Python's **json** module converts between JSON text and Python objects.

Function	Purpose
`json.loads(s)`	JSON **string** → Python object (dict / list)
`json.dumps(obj)`	Python object → JSON **string** (use *indent=2* for neat output)
`json.load(f)`	Read JSON from a **file** → Python object
`json.dump(obj, f)`	Write a Python object to a **file** as JSON

JSON	object { }	array [ ]	string "..."	number	true / false	null
Python	dict	list	str	int / float	True / False	None

```
import json
text = '{"name": "Asha", "marks": [90, 85], "passed": true, "address": null}'
data = json.loads(text)                  # JSON string -> Python dict
print(data["name"], data["marks"][0], data["passed"], data["address"])
print(json.dumps(data, indent=2))        # Python -> JSON string
```

Output

```
Asha 90 True None
{
  "name": "Asha",
  "marks": [
    90,
    85
  ],
  "passed": true,
  "address": null
}
```

4. XML in Python

XML (eXtensible Markup Language) stores data in a tree of **tags** (elements) with attributes and text. Python reads it with **xml.etree.ElementTree**.

ElementTree item	Meaning
`ET.parse("f.xml")`	Reads an XML file; use *.getroot()* to get the root element
`ET.fromstring(s)`	Parses an XML string and returns the root element
`elem.tag / elem.text`	Tag name and the text inside the element
`elem.get("id") / elem.attrib`	Attribute value / all attributes as a dictionary
`elem.find("t") / findall("t")`	First matching child / list of all matching children
`ET.SubElement(parent, "t")`	Creates a new child element (to build XML)

```
import xml.etree.ElementTree as ET
xml_data = """<students>
  <student id="1"><name>Asha</name><marks>90</marks></student>
  <student id="2"><name>Ravi</name><marks>75</marks></student>
</students>"""
root = ET.fromstring(xml_data)
print(root.tag)
for s in root.findall("student"):
    print(s.get("id"), s.find("name").text, s.find("marks").text)
```

Output

```
students
1 Asha 90
2 Ravi 75
```

	JSON	XML
Syntax	Key-value pairs, braces and brackets	Opening and closing tags
Size / speed	Shorter and faster to parse	Longer (more verbose)
Data types	Numbers, strings, booleans, arrays	Everything is text
Common use	Web APIs, configuration	Documents, older systems, RSS, SOAP

5. NumPy with Python

NumPy (Numerical Python) provides the **ndarray** — a fast, fixed-type, multi-dimensional array. It is much faster than Python lists and allows **vectorised** operations (apply an operation to the whole array without loops).

	Python list	NumPy array
Data types	Can mix types	All elements have **one** dtype
Speed / memory	Slower, more memory	Much faster, compact
*[1,2]*2 / a*2*	Repeats the list → [1,2,1,2]	Multiplies each element → [2,4]

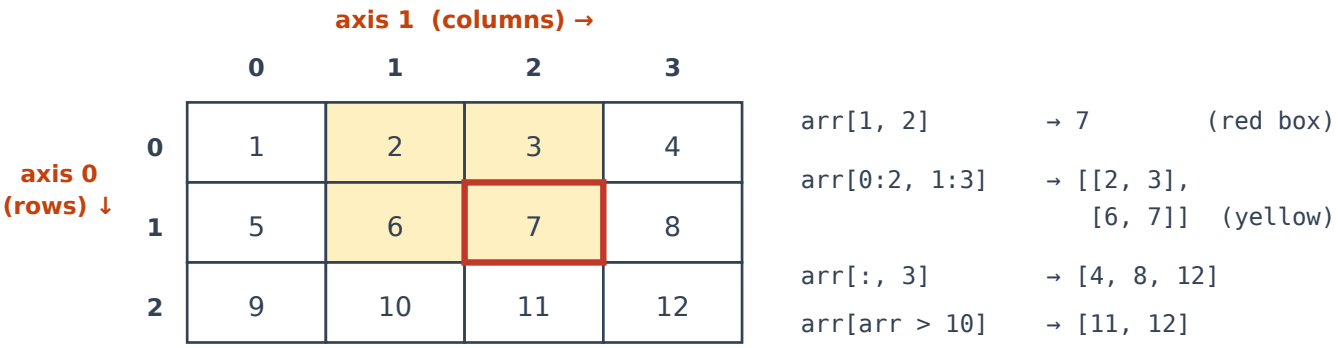
Creating arrays

Function	Result
`np.array([1,2,3])`	Array from a list / tuple (the main way)
`np.zeros((2,3)) np.ones((2,3))`	Arrays filled with 0 / 1
`np.arange(0, 10, 2)`	Like range(): 0,2,4,6,8
`np.linspace(0, 1, 5)`	5 equally spaced numbers from 0 to 1
`np.eye(3)`	Identity matrix
`np.random.rand(2,2)`	Random numbers between 0 and 1

Important attributes

Attribute	Meaning
`a.ndim`	Number of dimensions (axes)
`a.shape`	Tuple of size in each dimension, e.g. (3, 4) = 3 rows, 4 columns
`a.size`	Total number of elements
`a.dtype`	Data type of the elements (int64, float64 …)

Indexing and slicing




```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(arr.shape, arr.ndim, arr.size, arr.dtype)
print(arr[1, 2])                  # row 1, column 2
print(arr[0:2, 1:3])              # slicing: rows 0-1, columns 1-2
print(arr[:, 3])                  # whole last column
print(arr[arr > 10])              # Boolean indexing
print(arr * 2)                    # vectorised operation
print(arr.reshape(4, 3).shape)    # change shape
```

Output

```
(3, 4) 2 12 int64
7
[[2 3]
 [6 7]]
[ 4  8 12]
[11 12]
[[ 2  4  6  8]
 [10 12 14 16]
 [18 20 22 24]]
(4, 3)
```

Aggregate functions

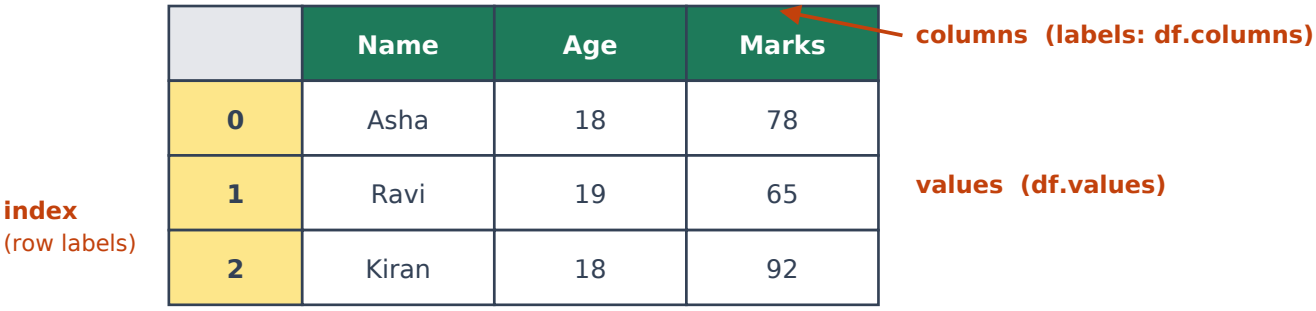
Function	Result	Function	Result
`a.min()`	Smallest value	`a.sum()`	Total of all elements
`a.max()`	Largest value	`a.cumsum()`	Running (cumulative) sum
`a.mean()`	Average	`a.sum(axis=0)`	Column-wise total (axis=1 → row-wise)

★ Study Tip

- Remember **axis 0 = rows direction (down)**, **axis 1 = columns direction (across)**.
- Slicing *a[start:stop]* gives a **view** (no copy). Use *.copy()* if you need an independent array.

6. Pandas

Pandas is built on NumPy and gives two main structures: **Series** (one-dimensional labelled array) and **DataFrame** (two-dimensional table with rows and columns, like an Excel sheet).



	Name	Age	Marks
0	Asha	18	78
1	Ravi	19	65
2	Kiran	18	92

DataFrame: every column is a Series with its own data type

Creating and exploring

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["a", "b", "c"])     # Series
print(s["b"])

df = pd.DataFrame({"Name": ["Asha", "Ravi", "Kiran"],
                   "Age": [18, 19, 18],
                   "Marks": [78, 65, 92]})              # from a dictionary
print(df)
print(df.shape, list(df.columns))
print(df.describe())                                    # quick statistics
```

Output

```
20
    Name  Age  Marks
0   Asha   18     78
1   Ravi   19     65
2  Kiran   18     92
(3, 3) ['Name', 'Age', 'Marks']
             Age      Marks
count   3.000000   3.000000
mean   18.333333  78.333333
std     0.577350  13.503086
min    18.000000  65.000000
25%    18.000000  71.500000
50%    18.000000  78.000000
75%    18.500000  85.000000
max    19.000000  92.000000
```

Selecting data

Operation	Code	Gives
One column	`df["Marks"]`	A Series
Many columns	`df[["Name","Marks"]]`	A DataFrame
Row by label	`df.loc[1]`	Row whose index label is 1
By position	`df.iloc[0:2, 0:2]`	First 2 rows and first 2 columns
By condition	`df[df["Marks"] > 70]`	Rows where Marks > 70
First / last rows	`df.head(n) df.tail(n)`	First / last n rows (default 5)

```
import pandas as pd
df = pd.DataFrame({"Name": ["Asha", "Ravi", "Kiran"], "Age": [18, 19, 18], "Marks": [78, 65, 92]})
print(df[df["Marks"] > 70])                 # filter rows
df["Result"] = df["Marks"] >= 70            # add a new column
print(df.sort_values("Marks", ascending=False).head(2))
df.to_csv("marks.csv", index=False)         # save to CSV
```

```
print(pd.read_csv("marks.csv").shape)       # read it back (rows, columns)
```

Output

```
    Name  Age  Marks
0   Asha   18     78
2  Kiran   18     92
    Name  Age  Marks  Result
2  Kiran   18     92    True
0   Asha   18     78    True
(3, 4)
```

i Remember

- **loc** uses **labels**, **iloc** uses **integer positions**. Missing data: *df.isnull()*, *df.dropna()*, *df.fillna(value)*.

7. Plotting with Matplotlib

Matplotlib draws graphs. The common functions are in **matplotlib.pyplot** (imported as *plt*).

Function	Purpose
`plt.plot(x, y)`	Line graph — shows how y changes with x
`plt.scatter(x, y)`	Scatter plot — one dot per (x, y) pair, shows relationship / correlation
`plt.bar(x, h) plt.hist(d)`	Bar chart / histogram
`plt.xlabel() ylabel() title()`	Axis labels and title
`plt.show()`	Displays the figure

✓ Quick Revision

- **Functional**: lambda (anonymous function), map (apply), filter (select), reduce (combine, from functools), comprehension.
- **JSON**: loads/dumps (strings), load/dump (files). object→dict, array→list, true→True, null→None.
- **XML**: ElementTree — fromstring/parse, find/findall, .text, .get(attr).
- **NumPy**: np.array(); ndim, shape, size, dtype; slicing a[r1:r2, c1:c2]; Boolean index a[a>5]; min/max/sum/cumsum.
- **Pandas**: Series (1-D), DataFrame (2-D); head(); loc (label) / iloc (position); df[condition]; read_csv / to_csv.
- **Matplotlib**: plot (line), scatter (dots), labels, show().

SAMPLE PROGRAMS 24 – 30 (with output)

Program 24

Question

- Python program to check whether a JSON string contains complex object or not.

Program

```
import json

def has_complex(data):
    # complex object = a value that is itself a dictionary (object) or list (array)
    if isinstance(data, dict):
        values = data.values()
    elif isinstance(data, list):
        values = data
    else:
        return False
    return any(isinstance(v, (dict, list)) for v in values)

s1 = '{"name": "Ravi", "age": 20}'
s2 = '{"name": "Ravi", "marks": {"maths": 90, "physics": 85}}'
s3 = '{"name": "Ravi", "subjects": ["Maths", "Physics"]}'

for s in (s1, s2, s3):
    data = json.loads(s)                    # JSON string -> Python object
    result = "Complex object" if has_complex(data) else "Simple (no complex object)"
    print(s, "->", result)
```

Output

```
{"name": "Ravi", "age": 20} -> Simple (no complex object)
{"name": "Ravi", "marks": {"maths": 90, "physics": 85}} -> Complex object
{"name": "Ravi", "subjects": ["Maths", "Physics"]} -> Complex object
```

i Remember

- A JSON value is **simple** if it is a string, number, boolean or null. If it is an **object { }** or **array [ ]** then it is a **complex** (nested) value.

Program 25

Question

- Python program to demonstrate NumPy arrays creation using array() function.

Program

```
import numpy as np

a = np.array([10, 20, 30, 40])                      # 1-D array from a list
b = np.array([[1, 2, 3], [4, 5, 6]])                # 2-D array (list of lists)
c = np.array((7, 8, 9))                             # from a tuple
d = np.array([1, 2, 3], dtype=float)                # with a given data type
e = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # 3-D array

print("1-D array :", a)
print("2-D array :\n", b)
print("From tuple:", c)
print("Float type:", d)
```

```
print("3-D array :\n", e)
```

Output

```
1-D array : [10 20 30 40]
2-D array :
 [[1 2 3]
 [4 5 6]]
From tuple: [7 8 9]
Float type: [1. 2. 3.]
3-D array :
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Program 26

Question

- Python program to demonstrate use of ndim, shape, size, dtype.

Program

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr)
print("ndim  (dimensions)  :", arr.ndim)
print("shape (rows, cols) :", arr.shape)
print("size  (elements)   :", arr.size)
print("dtype (data type)  :", arr.dtype)

f = np.array([1.5, 2.5, 3.5])
print("Float array dtype  :", f.dtype)
g = f.astype(int)                       # change the data type
print("After astype(int)  :", g, g.dtype)
```

Output

```
Array:
 [[1 2 3]
 [4 5 6]]
ndim  (dimensions)  : 2
shape (rows, cols) : (2, 3)
size  (elements)   : 6
dtype (data type)  : int64
Float array dtype  : float64
After astype(int)  : [1 2 3] int64
```

i Remember

- The integer dtype may print as **int32** on some Windows systems and **int64** on others — both are correct.

Program 27

Question

- Python program to demonstrate basic slicing, integer and Boolean indexing.

Program

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

print("Basic slicing")
print("arr[0:2, 1:3] =\n", arr[0:2, 1:3])    # rows 0-1, columns 1-2
print("arr[:, 2]     =", arr[:, 2])           # all rows, column 2
print("arr[1]        =", arr[1])              # row 1

print("\nInteger indexing")
print("arr[1, 2]              =", arr[1, 2])
print("arr[[0, 1, 2], [1, 2, 3]] =", arr[[0, 1, 2], [1, 2, 3]])  # picks (0,1),(1,2),(2,3)

print("\nBoolean indexing")
print("arr > 6        =\n", arr > 6)
print("arr[arr > 6]   =", arr[arr > 6])
print("arr[arr % 2 == 0] =", arr[arr % 2 == 0])
```

Output

```
Basic slicing
arr[0:2, 1:3] =
 [[2 3]
 [6 7]]
arr[:, 2]     = [ 3  7 11]
arr[1]        = [5 6 7 8]

Integer indexing
arr[1, 2]              = 7
arr[[0, 1, 2], [1, 2, 3]] = [ 2  7 12]

Boolean indexing
arr > 6        =
 [[False False False False]
 [False False  True  True]
 [ True  True  True  True]]
arr[arr > 6]   = [ 7  8  9 10 11 12]
arr[arr % 2 == 0] = [ 2  4  6  8 10 12]
```

Program 28

Question

- Python program to find min, max, sum, cumulative sum of array.

Program

```
import numpy as np

a = np.array([4, 7, 1, 9, 3])
print("Array     :", a)
print("Minimum   :", a.min(), " at index", a.argmin())
print("Maximum   :", a.max(), " at index", a.argmax())
print("Sum       :", a.sum())
print("Cumulative sum:", a.cumsum())
```

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print("\n2-D array:\n", b)
print("Column-wise sum (axis=0):", b.sum(axis=0))
print("Row-wise sum    (axis=1):", b.sum(axis=1))
print("Cumulative sum of all   :", b.cumsum())
```

Output

```
Array     : [4 7 1 9 3]
Minimum   : 1  at index 2
Maximum   : 9  at index 3
Sum       : 24
Cumulative sum: [ 4 11 12 21 24]

2-D array:
 [[1 2 3]
 [4 5 6]]
Column-wise sum (axis=0): [5 7 9]
Row-wise sum    (axis=1): [ 6 15]
Cumulative sum of all   : [ 1  3  6 10 15 21]
```

Program 29

Question

- Create a dictionary with at least five keys and each key represent value as a list where this list contains at least ten values and convert this dictionary as a pandas data frame and explore the data through the data frame as follows: a) Apply head() function to the pandas data frame b) Perform various data selection operations on Data Frame.

Program

```
import pandas as pd

data = {
    "Roll": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Name": ["Asha","Ravi","Kiran","Divya","Eswar","Farah","Gopi","Hema","Imran","Jaya"],
    "Age": [18, 19, 18, 20, 19, 18, 20, 19, 18, 19],
    "Marks": [78, 65, 92, 55, 88, 70, 45, 81, 60, 95],
    "Attendance": [85, 72, 95, 60, 90, 78, 50, 88, 68, 97],
}

df = pd.DataFrame(data)                  # dictionary -> DataFrame
print("Full DataFrame:\n", df)

print("\n(a) head() - first 5 rows:\n", df.head())
print("\nhead(3):\n", df.head(3))

print("\n(b) Data selection operations")
print("\n1. Single column  df['Name']:")
print(df["Name"].head(3))
print("\n2. Multiple columns  df[['Name', 'Marks']]:")
print(df[["Name", "Marks"]].head(3))
print("\n3. Row by label  df.loc[2]:")
print(df.loc[2])
print("\n4. By position  df.iloc[0:3, 1:4]:")
print(df.iloc[0:3, 1:4])
print("\n5. Condition  df[df['Marks'] > 80]:")
print(df[df["Marks"] > 80])
print("\n6. Rows and columns together  df.loc[df['Age'] == 18, ['Name', 'Marks']]:")
print(df.loc[df["Age"] == 18, ["Name", "Marks"]])
```

Output

```
Full DataFrame:
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95
3     4  Divya   20     55          60
4     5  Eswar   19     88          90
5     6  Farah   18     70          78
6     7   Gopi   20     45          50
7     8   Hema   19     81          88
8     9  Imran   18     60          68
9    10   Jaya   19     95          97

(a) head() - first 5 rows:
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95
3     4  Divya   20     55          60
4     5  Eswar   19     88          90

head(3):
    Roll   Name  Age  Marks  Attendance
0     1   Asha   18     78          85
1     2   Ravi   19     65          72
2     3  Kiran   18     92          95

(b) Data selection operations

1. Single column  df['Name']:
0     Asha
1     Ravi
2    Kiran
Name: Name, dtype: str

2. Multiple columns  df[['Name', 'Marks']]:
    Name  Marks
0   Asha     78
1   Ravi     65
2  Kiran     92

3. Row by label  df.loc[2]:
Roll              3
Name          Kiran
Age              18
Marks            92
Attendance       95
Name: 2, dtype: object

4. By position  df.iloc[0:3, 1:4]:
    Name  Age  Marks
0   Asha   18     78
1   Ravi   19     65
2  Kiran   18     92

5. Condition  df[df['Marks'] > 80]:
   Roll   Name  Age  Marks  Attendance
2     3  Kiran   18     92          95
4     5  Eswar   19     88          90
7     8   Hema   19     81          88
9    10   Jaya   19     95          97

6. Rows and columns together  df.loc[df['Age'] == 18, ['Name', 'Marks']]:
    Name  Marks
0   Asha     78
2  Kiran     92
5  Farah     70
8  Imran     60
```

Program 30

Question

- Select any two columns from the above data frame, and observe the change in one attribute with respect to other attribute with scatter and plot operations in matplotlib.

Program

```
import pandas as pd

data = {
    "Roll": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Name": ["Asha","Ravi","Kiran","Divya","Eswar","Farah","Gopi","Hema","Imran","Jaya"],
    "Age": [18, 19, 18, 20, 19, 18, 20, 19, 18, 19],
    "Marks": [78, 65, 92, 55, 88, 70, 45, 81, 60, 95],
    "Attendance": [85, 72, 95, 60, 90, 78, 50, 88, 68, 97],
}

import matplotlib.pyplot as plt

df = pd.DataFrame(data).sort_values("Attendance")   # sort so the line graph is smooth
x = df["Attendance"]                                # attribute 1
y = df["Marks"]                                     # attribute 2

fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].scatter(x, y, color="crimson")                # scatter operation
ax[0].set_title("Scatter plot")
ax[0].set_xlabel("Attendance (%)")
ax[0].set_ylabel("Marks")

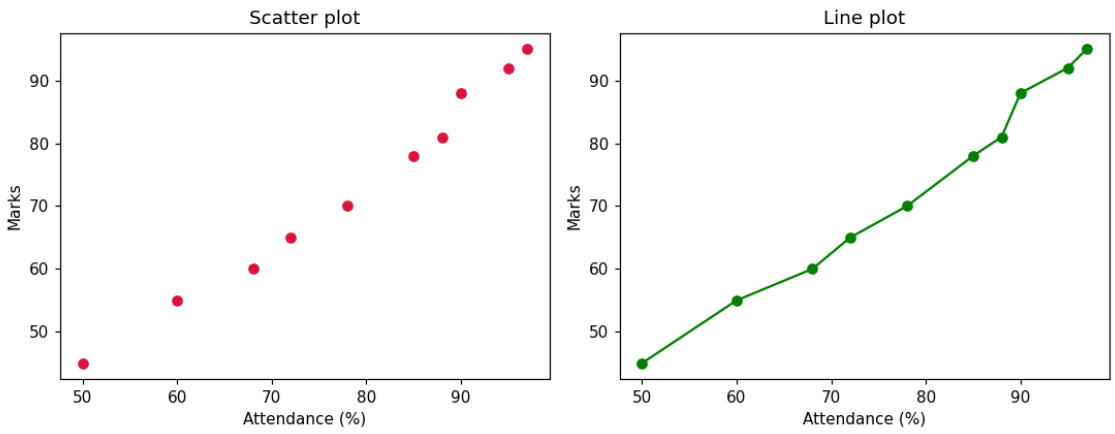
ax[1].plot(x, y, marker="o", color="green")         # plot (line) operation
ax[1].set_title("Line plot")
ax[1].set_xlabel("Attendance (%)")
ax[1].set_ylabel("Marks")

plt.tight_layout()
plt.show()
print("Correlation between Attendance and Marks:", round(x.corr(y), 3))
```

Output

```
Correlation between Attendance and Marks: 0.992
```

Graph produced



i Remember

- **Observation:** the scatter points and the line both rise from left to right — as **Attendance increases, Marks also increase** (strong positive correlation).

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a lambda function? 2. Differentiate map() and filter(). 3. What is JSON? Name any two json functions. 4. Differentiate JSON and XML. 5. What is a NumPy array? List any four attributes. 6. What is a DataFrame? 7. Differentiate loc and iloc. 8. What is the use of head()?	1. Explain functional programming with map, filter, reduce and lambda. 2. Explain JSON and XML handling in Python with examples. 3. Explain NumPy array creation, indexing and slicing with programs. 4. Explain Pandas Series and DataFrame with data selection operations. 5. Explain scatter and line plots using Matplotlib. 6. Write programs 24–30 from the sample experiments.